Character Information in Language Models: A Prompt-Independent Word Store and a Late, Question-Driven Signal That Grows With Scale*

Pratyush Acharya[1†] Nuraj Rimal[2] Apar Baral[3]
Habish Dhakal[1] Piyush Mohan Bhattarai[4]

[1]SecurityPal AI [2]Southern Illinois University Edwardsville
[3]University of Maryland [4]Saint Louis University

October 15, 2026

Abstract

Linear probes recover character properties of a word from a language model's hidden states, but a probe at the answer position cannot say whether the property was stored with the word or produced because the prompt asked for it. We separate the two with a word-position baseline: the same held-out-word probe applied to the query word's own tokens in a sentence that asks nothing. Across 16 open-weight models from seven families (0.6B–14B) and eleven character tasks, the word tokens encode `first_letter`, `last_letter` and `most_frequent_letter` above a word-only nuisance floor in all 16 models, and about 70% of that signal is already present in the token embedding. Asking the question adds a second signal at the answer position. Its size grows with model scale for all four prompt-sensitive tasks (Holm-corrected permutation $p \leq 0.015$, also within model families), and the growth is confined to the late layers: restricted to the first quarter of the network, the slope is indistinguishable from zero. Small models' answer positions hold less than their word-level store; large models' hold more. Three controls bound these claims. Measured against the nuisance floor, the question-free signal that a majority-baseline analysis counts as retained largely disappears (`last_letter`: -0.01). Count tasks become measurable only after length, repetition and query-letter shortcuts are removed by design. A pre-registered test finds that a question placed before the word does not change the word's own encoding beyond what an arbitrary prefix change does. Representation–readout gaps show no robust scaling once probe saturation is accounted for. Pre-registered criteria, a fixed estimator (held-out layer selection, cold-start folds) and all analysis code accompany the paper.

1 Introduction

Linear probes often recover a correct answer from a language model's hidden states when the model's own output is wrong. This representation–output gap has been studied for factual and character-level tasks [Park et al., 2026, Datta et al., 2026, Gekhman et al., 2025, Hiraoka and Inui, 2025], and it is often read as evidence that the model has the answer but fails to express it. For character properties that reading skips a step. The last letter of a word is a fixed property of the word, and subword embeddings are known to encode spelling [Kaushal and Mahowald, 2022, Itzhak

*Code and data: https://github.com/pratyush-acharya/char-store-probing

†Corresponding author: pratyush.acharya@securitypalhq.com

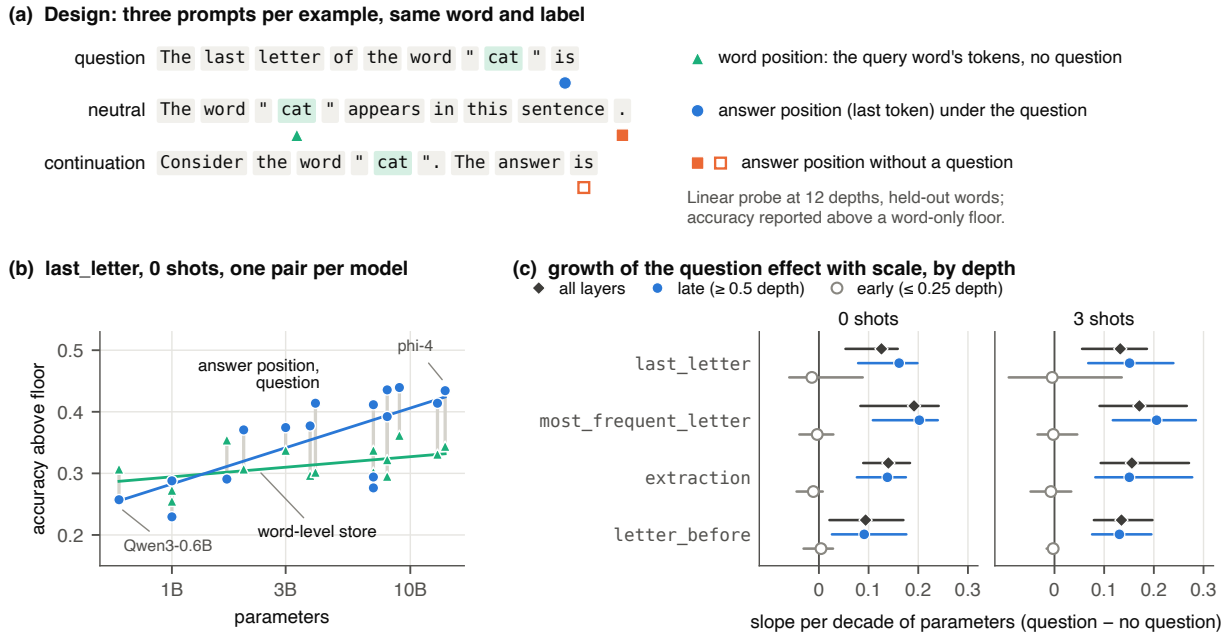


Figure 1: Overview. (a) Each example is run as three prompts with the same word and label. A linear probe, trained and tested on disjoint words, reads the query word's own tokens in the neutral sentence (the word-level store) and the final token of each prompt (the answer position). (b) `last_letter` at zero shots, one pair of points per model: the word-level store (last-token read) and the answer position under the question, both above the word-only floor, with OLS lines. The answer position lies below the store in the smallest models and above it in the largest (§5). (c) Slope per decade of parameters of the question − no-question difference at the answer position for the four prompt-sensitive tasks, with 95% intervals from resampling model families: over all layers (held-out layer selection) and restricted to late (≥ 0.5) or early (≤ 0.25) relative depth. Slopes, permutation p-values and their Holm-corrected values are in Table 4.

and Levy, 2022]. A probe at the answer position may therefore read a property that was stored with the word, one that the question caused the model to assemble, or both.

We separate these with a *word-position baseline*: the same held-out-word probe, applied to the query word's own tokens inside a sentence that asks nothing (`The word "cat" appears in this sentence.`). Under causal attention those tokens have seen only `The word "`, so their decodability is the closest available measure of what the model stores about the word independently of any task. We compare it with the answer position under the task question and under two question-free carriers, for 16 open-weight models from seven families (0.6B–14B) and eleven character tasks, using held-out-word cross-validation, a word-only nuisance floor, held-out layer selection, and pre-registered decision criteria.

Three results organize the paper.

A prompt-independent store exists, and it is mostly lexical. The word tokens encode `first_letter`, `last_letter` and `most_frequent_letter` above the nuisance floor in all 16 models (§4). Roughly 70% of that above-floor signal is already present at layer 0, in the token embedding, consistent with prior work on spelling in embeddings. This half of the picture is confirmation, not discovery, but it supplies the baseline that the rest of the analysis needs.

Asking adds a late signal, and only that signal grows with scale. At the answer position, the difference between the question prompt and the question-free carrier grows with log parameter

count for all four prompt-sensitive tasks, after Holm correction and within model families (§5; Figure 1c). Restricting the probe to layers past half depth leaves the effect intact; restricting it to the first quarter removes it. The answer position of small models holds less than their word-level store, while that of large models holds more (Figure 1b). Layer profiles show the two phases directly (Figure 4): an early bump that is the same at every scale, then a late rise that appears only in larger models.

Controls change what can be claimed. Measured against the word-only floor rather than the majority class, most of the question-free signal that a majority-baseline analysis counts as retained is nuisance (§5). Count tasks fail the floor until length, letter repetition and query-letter identity are removed by design (§7). Representation–readout gaps do not follow a robust scaling trend once probe saturation is accounted for (§8). A pre-registered stem-order experiment finds that placing the question before the word does not change the word's encoding beyond what an arbitrary two-word prefix change does (Appendix B).

Contributions. (1) A word-position baseline that separates prompt-independent storage from question-conditioned decodability at the answer position. (2) Evidence that the scale-dependent part of character decodability is late and question-driven, while early and word-level signals are scale-invariant. (3) A set of controls (word-only floors with designed-out shortcuts, held-out layer selection, pre-registration) that each reversed or removed a claim an uncontrolled analysis would have made. We do not identify the mechanism behind the late signal; whether it reflects computation, integration across the word's tokens, or reformatting is left open.

2 Related Work

Character information in subword models. Token embeddings of pretrained models encode much of their tokens' spelling, and this improves with model size [Kaushal and Mahowald, 2022]; the embedding layers of RoBERTa and GPT-2 suffice to spell up to a third of the vocabulary [Itzhak and Levy, 2022]. Controlled training comparisons attribute the acquisition of such knowledge to tokenization-related factors (merge rules, orthographic constraints) and to semantic and syntactic factors independent of tokenization [Sato and Sasano, 2026]. Character tasks nonetheless remain hard at the output: the Strawberry Problem traces the emergence of character-level performance during training [Cosma et al., 2025], CharBench identifies word length and character count as predictors of errors [Uzan and Pinter, 2026], and stochastic segmentation improves fine-grained subword processing [Sims et al., 2025]. Our word-position baseline connects these lines: it measures the stored information at the word in context and asks what the question adds at the answer position.

Representation–output discrepancies. Linear or diagnostic readouts can recover information a model does not express, for multiple-choice questions [Park et al., 2026], hidden factual knowledge [Gekhman et al., 2025] and character tasks compared layer by layer [Hiraoka and Inui, 2025]. Mechanistic studies of counting report early encoding followed by late suppression [Datta et al., 2026] and a format-sensitive late MLP block that overwrites a probe-correct repeated-token count [Venkatesh, 2026]. Across cultural knowledge, probe–output discrepancies decrease with scale but persist within families [Chelombitko et al., 2026]. We find that character-level discrepancies do not share a single scaling trend once saturation is accounted for.

Probe interpretation. Probe accuracy depends on the representation, probe class, data and split; control tasks and selectivity are therefore central [Hewitt and Liang, 2019, Belinkov, 2022]. Vocabulary overlap permits word-level memorization [Rosa et al., 2020, Dhar and Bisazza, 2020], and the amount of probe data changes conclusions [Voita and Titov, 2020, Zhu et al., 2022]. Datta et al. [2026] omit control tasks on the grounds that their dataset construction prevents memorization; we record that position without adopting it, and use grouped splits by query word and word-only nuisance floors instead. We do not run a random-label selectivity control.

Prompts, demonstrations and readout geometry. Demonstrations convey label space and format even with degraded input–label mappings [Min et al., 2022]; task recognition and task learning can be separated [Pan et al., 2023], and scale changes in-context behaviour [Wei et al., 2023]. Our zero-shot contrasts remove demonstrations from every arm. Embedding-space analyses and tuned lenses relate hidden states to the output vocabulary [Dar et al., 2023, Belrose et al., 2023], and amnesic probing removes a learned subspace [Elazar et al., 2021]. A counting study finds a count direction nearly orthogonal to each digit-token row [Garcia, 2026]; we test the joint span of answer-token rows, a different object. Inference-time interventions can improve selected outputs [Li et al., 2023]; we make no causal claim.

3 Experimental Setup

Tasks. Nine character tasks over English words of length 3–9: `first_letter`, `last_letter`, `contains_letter` (yes/no), `extraction` (k-th letter), `counting` (occurrences of a queried letter), `most_frequent_letter`, `letter_before` (the letter preceding a queried letter), `word_length` and `count_distinct`. Two further tasks, `counting_lm` and `count_distinct_lm`, ask the same questions as the two count tasks but sample examples so that word-only shortcuts carry no information about the answer (§7, Appendix A.3). Five tasks (`first_letter`, `last_letter`, `most_frequent_letter`, `word_length`, `count_distinct`) are pure functions of the word; the others carry a query letter or position. Prompts and answer sets are in Appendix A.2.

Stimuli and models. Each task has 800 examples sampled from a 6000-word frequency-ranked English vocabulary built from the Brown Corpus [Kučera and Francis, 1967], realizing 405–757 unique words per task. We evaluate 16 open-weight models from seven families: Qwen3 (0.6/1.7/4/8B), Llama-3.x (1/3/8B), Gemma-2 (2/9B), OLMo-2 (1/7/13B), Phi-4 (3.8B-mini, 14B), OLMo-Hybrid-7B and OLMoE-1B-7B [Yang et al., 2025, Grattafiori et al., 2024, Gemma Team, 2024, Abdin et al., 2024, Team OLMo et al., 2025, Muennighoff et al., 2025, Merrill et al., 2026]; Table 2 (Appendix D) lists them. Six are post-trained (the four Qwen3 releases and both Phi-4 models); we report base-only statistics over the other ten where post-training could matter. All run in bf16 on one 48 GB GPU. Prompts are completion-style, at 0 or 3 demonstrations. At zero shots no arm contains a demonstration, which makes it the clean contrast; most claims are reported at both counts.

Prompt arms and positions. For every example we run three prompts on the same word and label (Figure 1a): the **question**, the task prompt (e.g. `The last letter of the word "cat" is`); a **neutral** sentence, `The word "cat" appears in this sentence.`; and a **continuation** carrier, `Consider the word "cat". The answer is`, which matches the question's "awaiting an answer" final position without naming the property. Term tasks carry their query letter or position into both carriers. We read hidden states at two positions: the **answer position** (the final prompt

token) in every arm, and the **word position** in the question and neutral arms, for the pure-word tasks, as either the word's last token or the mean over all tokens overlapping the word.

Probe and estimator. A standardized multinomial logistic regression ($\ell_2 = 10^{-3}$) is fit at each of 12 evenly spaced layers (including the embedding layer and the last layer), with three-fold cross-validation grouped by query word, so no query word appears in both training and test labels. Folds are cold-started. The reported accuracy uses *held-out layer selection*: for each test fold, the layer is chosen on the other folds. With this estimator, the peak-over-layers accuracy exceeds the held-out-selected accuracy by 0.001–0.019 per task, with no positive dependence on model depth, and all representation numbers lie within ±0.021 of an absolute-stride, warm-started re-estimate (Appendix A.4).

Baselines. The **majority** baseline predicts the training fold's modal answer. The **nuisance floor** is the best of several predictors that see only the word: its length, its position-free letter set, the query letter where the task has one, and their combination, fit on the same folds. Predictors that are sufficient for the label (length for `word_length`, the letter set for `count_distinct`) are excluded. We report accuracy *above the floor* unless stated otherwise.

Readout. The model's own accuracy is the argmax of its next-token logits over the answer-token set (26 letters, yes/no, or ten digits). At zero shots, completion prompts are a degenerate format for readout: larger models continue the sentence rather than answer, so readout claims use three shots.

Statistics. Scaling associations are Pearson correlations or OLS slopes against $\log_{10}$ parameters, with two-sided permutation p-values from 10,000 permutations of model size. Because each model contributes every arm, a question×scale interaction equals the slope of the per-model difference between arms. We also report a within-family (fixed-effect) slope, a 95% interval from resampling families, and base-only correlations. Where a claim spans the four prompt-sensitive tasks we apply Holm correction [Holm, 1979]. The 16 checkpoints are related and not independent draws; these are cross-model associations, not population scaling laws.

4 A Prompt-Independent Word-Level Store

The store is present in every model. The word tokens encode `first_letter`, `last_letter` and `most_frequent_letter` above the floor in all 16 models, at both shot counts and under both reads (Figure 2a; exact values and model counts in Table 3, Appendix D). The size depends on where the property lives in the word: the last-token read is stronger for `last_letter` (+0.314 against +0.177 for the span mean at zero shots), and the span mean is stronger for `first_letter` and `most_frequent_letter`. We take the better read per task as the store in what follows; every conclusion below also holds under the other read.

It is mostly in the embedding. At layer 0 (Figure 2b) the word tokens already carry 73% of `last_letter`'s above-floor store (last-token read, zero shots; 69% at three shots) and 82% of `most_frequent_letter`'s (span mean; 75% at three shots). For `first_letter` the share is 53–56%, with the rest added by intermediate layers. This agrees with direct evidence that token embeddings encode spelling [Kaushal and Mahowald, 2022, Itzhak and Levy, 2022]. In the present setting it means that the prompt-independent store is largely lexical: it is a property of the tokens, read off with little contextual processing.

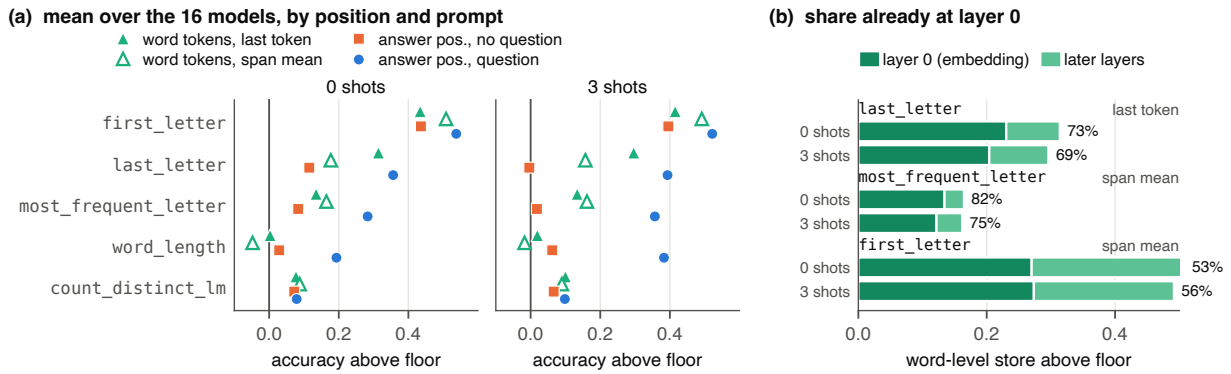


Figure 2: The word-level store. (a) Accuracy above the word-only floor (held-out layer selection), mean over the 16 models: the query word's tokens in the neutral sentence (last token, or mean over the word's tokens) and the final prompt token without and with the question. For `count_distinct_lm` the floor equals the majority baseline. (b) Part of the word-level store (better read per task) already present at layer 0, the token embedding; the percentage is that part's share of the store.

It does not grow reliably with scale. Across the 16 models (Figures 1b and 3) the store's correlation with $\log_{10}$ parameters is $r = 0.47$ ($p = 0.064$) for `last_letter`, $r = -0.05$ for `most_frequent_letter` and $r = -0.16$ for `first_letter` (zero shots, last-token read; span-mean values are similar). By contrast, the question arm's answer position correlates with scale at $r = 0.73$ ($p = 0.002$) and $r = 0.72$ ($p = 0.002$) for the first two tasks.

Where there is no store. `word_length` is not linearly decodable at the word position above its floor, which here is the letter-set predictor (+0.003 last token, −0.048 span mean at zero shots). Its answer position under the question reaches +0.383 at three shots, so length, unlike the letter properties, is assembled at the answer position rather than read off the word. `count_distinct_lm` has a small store (+0.08 to +0.10 above majority in all 16 models) that the question barely changes.

Sensitivity to the preceding context. The store is measured after the prefix `The word "`. In a pre-registered follow-up (Appendix B) the prefix `In the word "` shifted the same word tokens' accuracy by +0.02 to +0.05, with the word tokenized identically. Prefix sensitivity of this size is small next to the store itself (+0.13 to +0.51 above the floor), but it sets the resolution at which word-level differences between prompts can be interpreted.

5 Asking Adds a Late Signal That Grows With Scale

The question effect grows with scale. Figure 3 shows the four prompt-sensitive tasks, the tasks whose question-free answer position retains little of the question arm's signal (below). Only the question arm rises with scale; the neutral arm is flat or falling, and so is the word-level store. The interaction is the slope of the per-model difference between the question and neutral arms (Figure 1c; Table 4 in Appendix D). It is positive for all four tasks at both shot counts, survives Holm correction across the four ($p_{\text{Holm}} \leq 0.015$), has a within-family slope of the same size, and has a family-resampling interval that excludes zero.

The growth is late. At zero shots the question arm's layer profile has two parts (Figure 4). In the first quarter of the network it shows a bump at about 10% depth that is similar in every model:

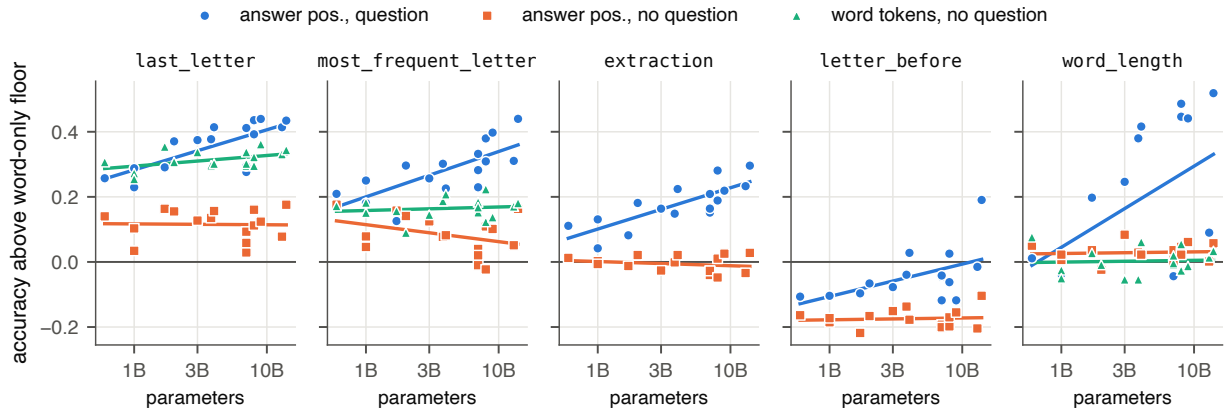


Figure 3: Accuracy above the word-only floor against model size, zero shots, one point per model with an OLS line per arm; the word tokens (triangles) are shown for the pure-word tasks (last-token read for `last_letter` and `word_length`, span mean for `most_frequent_letter`). Only the question arm rises with scale. The right panel contrasts a task with no word-level store: `word_length` is assembled at the answer position (§4). All values are in Table 3 and the released analysis files.

for `last_letter` the early-band accuracy averages 0.75 and does not vary with scale (slope −0.002 per decade, $p = 0.96$). One plausible source is the word's spelling being carried to the adjacent answer token. Past half depth, larger models rise again and smaller models do not. Restricting both arms to the late band leaves the interaction intact for every task ($p_{\text{Holm}} \leq 0.015$ at zero shots, ≤ 0.014 at three); restricting them to the early band removes it (all $|\text{slope}| \leq 0.014$, $p \geq 0.43$; Figure 1c). This also answers a confound: the question prompt's answer token sits next to the word, while the neutral sentence ends several tokens later. Distance could inflate the question arm early, but it is the same in every model and cannot produce a scale trend, and the trend is absent where distance would act.

Large models exceed their store; small models fall short of it. The answer position under the question does not simply relay the word-level store. For `last_letter` at zero shots, Qwen3-0.6B's answer position is below its own store (+0.257 against +0.307 above the floor), while phi-4's is above it (+0.434 against +0.344). The difference (answer position under the question minus store) grows with scale for both store tasks: +0.091 per decade ($p = 0.017$) at zero shots and +0.098 ($p = 0.0007$) at three for `last_letter`; +0.129 ($p = 0.012$) and +0.113 ($p = 0.0016$) for `most_frequent_letter`. The same holds within the late band. In the larger models the answer position therefore carries more linearly decodable information about the property than the word's own tokens do. Whether that excess reflects computation, integration across the word's tokens, or a change of format that a linear probe can exploit, our measurements do not say.

Where the late rise begins. As an exploratory description, the first depth past the early band at which the question arm exceeds the store becomes shallower with scale for `most_frequent_letter` at three shots ($r = -0.76$, $p = 0.0015$; median relative depth 0.77 in the smaller eight models, 0.59 in the larger eight). The other three task–shot cells lean the same way without reaching significance ($p = 0.06$–0.31), and 12 probed layers limit resolution to about 0.08 of depth, so we do not treat this as a finding.

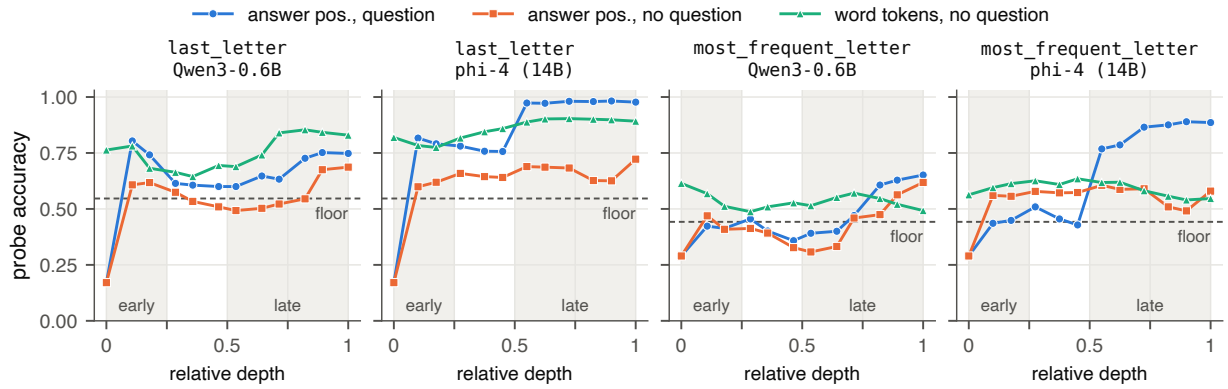


Figure 4: Layer profiles at zero shots for the smallest model (Qwen3-0.6B) and the largest (phi-4). Shaded bands mark the early (≤ 0.25) and late (≥ 0.5) depth ranges used in Figure 1c and Table 4; the dashed line is the word-only floor. The question arm's early bump is similar in both models; only the larger model's late phase rises well above the word-level store.

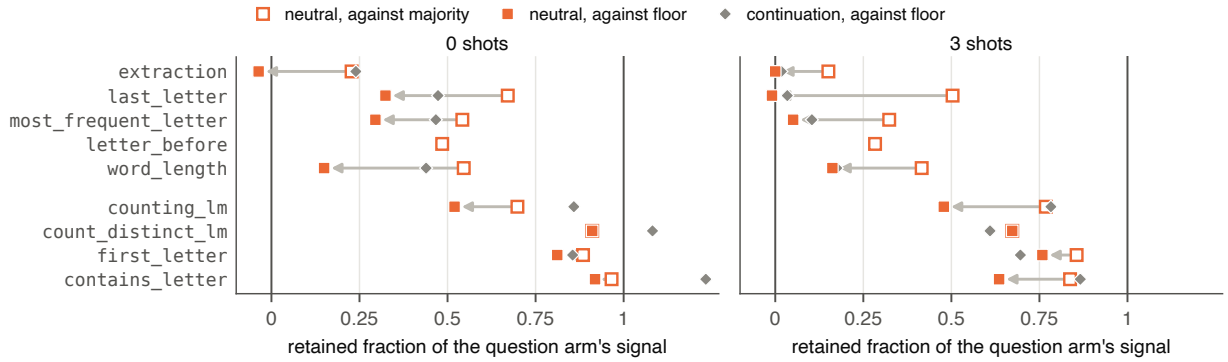


Figure 5: Retained fraction of the question arm's signal when the question is deleted: mean(no-question − base) / mean(question − base), at the answer position. Arrows run from the majority-based to the floor-based value for the neutral carrier; diamonds use the floor and the continuation carrier. `letter_before` has only the majority-based value, because its question arm's mean is not above the floor. Values in Table 5.

What survives deleting the question. *Retained* is the ratio of mean question-free accuracy to mean question accuracy, both measured above a baseline (Figure 5; Table 5). Against the majority class, `last_letter` retains half its signal at three shots (0.503). Against the word-only floor it retains none (−0.009, continuation carrier 0.034); `extraction` retains none and `most_frequent_letter` 5%. The majority-baseline figure was length and letter-set information that any word-level predictor has. At zero shots more survives (`last_letter` 0.32), consistent with the early bump, which reaches the sentence-final token of the neutral carrier only when no demonstrations intervene. `first_letter` and `contains_letter` retain 64–92% and are not question-driven; `word_length` retains 15–16% and joins the question-driven group.

6 How Specific Is the Question Effect?

To measure how much of the question effect of §5 depends on asking for *this* property, we probe label A while the prompt asks a different character question B about the same word, at zero shots, so that the self and cross prompts differ only in the question sentence. Primary cells pair a letter-valued

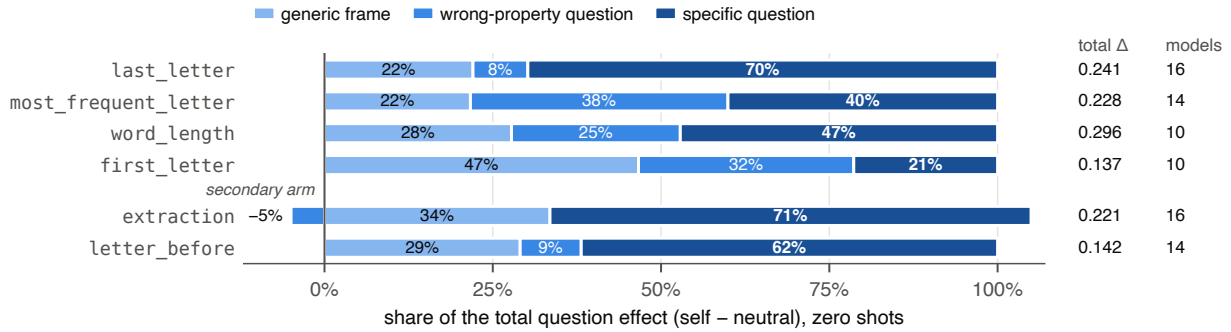


Figure 6: Decomposition of the question effect at zero shots, zero-leak cells, held-out layer selection: shares of the total Δ (self − neutral, above-majority scale; right), with a negative share drawn left of zero. All 16 models were run; "models" counts those with at least one qualifying cell. Below the gap: the secondary term-label arm. The continuation floor is joined per (model, set, task) from the word-position runs, whose example draws differ from the cross-task runs'. Values and cell counts in Table 6.

label with a digit-valued question or the reverse, so B's answer can never equal A's label; cells whose question arm is not at least 0.05 above the neutral arm are excluded. The two term-label tasks (`extraction`, `letter_before`) need their query term in every prompt, which we carry as a bare note line (`Note the 3rd position.`); they form a secondary arm because the prompt then differs in two elements.

We split the total question effect (self − neutral) into three additive parts: the generic frame of awaiting an answer (continuation − neutral), a wrong-property character question (cross − continuation), and the specific question (self − cross). Figure 6 shows the shares.

For `last_letter` and `extraction`, about 70% of the effect requires asking for the property itself; a wrong character question adds almost nothing (8% and −5%). `letter_before` is similar (62% specific, 9% wrong-property). For `most_frequent_letter`, a wrong character question already delivers 38% and the specific question 40%, so a substantial part of its effect is shared by character questions in general. `first_letter`, whose answer position is near ceiling without any question (§5), is dominated by the generic frame. Preliminary runs with a peak-over-layers, warm-started estimator gave 70%, 40% and 72% for `last_letter`, `most_frequent_letter` and `extraction`, within two percentage points of the present values.

Every cross prompt is itself a character question about the same word, so this design measures how specific the question effect is to the property asked. It does not separate a general character routine from property-specific routing, and we draw no conclusion about mechanism from it.

7 Word-Only Floors and the Count Tasks

A majority baseline controls for label skew, not for word properties that predict the label and that the model encodes. Figure 7 shows, for every task at three shots, how each model's question-arm probe compares with the word-only floor and the majority baseline (Table 7 gives the values).

The original count tasks fail. `counting` clears its floor in 2 of 16 models and `count_distinct` in 3. The cause is our own sampling. Stratifying `counting` by the true count made long words predict high counts, because a letter occurring three times needs a long word; length alone predicts the answer at 0.439 against a 0.311 majority. `count_distinct` has the same problem, since the number of distinct letters is bounded by word length (length alone: 0.532).

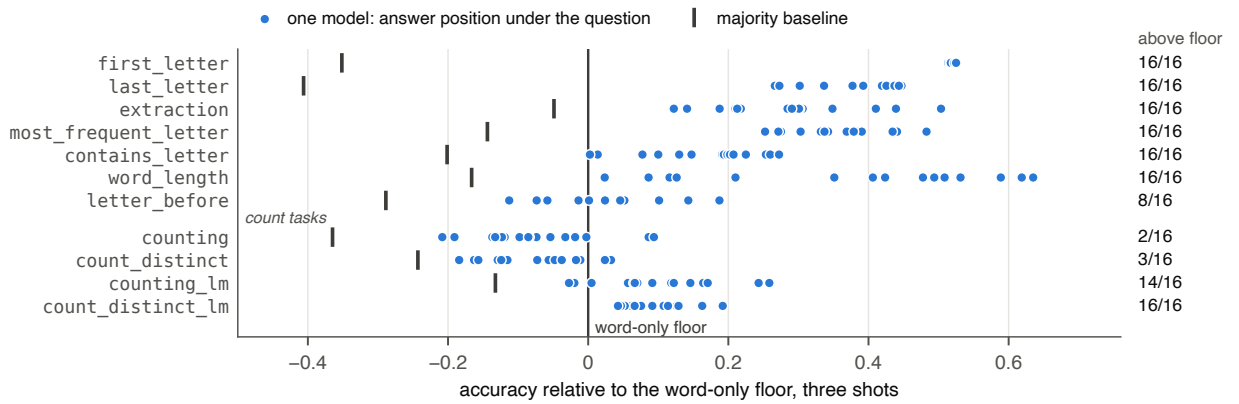


Figure 7: Word-only nuisance floors at three shots. Each dot is one model's question-arm probe (answer position, held-out layer selection), relative to the task's word-only floor (vertical line); the bar marks the majority baseline relative to the same floor (for `count_distinct_lm` the two coincide). Right: models above the floor. Below the gap: the original count tasks and their designed replacements. Floors, floor predictors and the best model per task are in Table 7.

Length matching is not enough. We first rebuilt the count tasks with equal numbers of examples in every (word length, answer) cell, which makes length statistically independent of the answer. For `counting` this left the floor at 0.61, because two other shortcuts remained. The query letter alone predicted the count at 0.46 (about half of the three-occurrence items query `e`), and the number of repeated letters (length minus distinct letters) predicted it at 0.59, since a word with no repeated letter can only give a count of one.

Designed-out shortcuts make the task measurable. `counting_lm` therefore also restricts words to exactly two repeated letters and draws the query letter uniformly within each cell from letters that occur at every count at that length (Appendix A.3). Length, repetition and query-letter predictors then sit near the majority baseline (0.25, 0.27 and 0.22 against 0.27), and the floor falls to 0.404, set by the letter set. `counting_lm` clears it in 14 of 16 models at three shots (16 of 16 at zero shots), by up to +0.259. `count_distinct_lm` draws equal numbers per cell over lengths 8–9 and answers 5–8; its only informative word predictor, the letter set, determines the answer and is excluded, so its floor equals the majority baseline and it tests above-majority decodability only.

The rebuilt count tasks are only partly question-driven. Without the question, `counting_lm` keeps about half of its above-floor signal (0.48–0.52, neutral carrier) and `count_distinct_lm` two thirds or more (0.67–0.91, Table 5). Neither shows a significant question×scale interaction (`counting_lm`: $p = 0.25$ and 0.12 at zero and three shots; `count_distinct_lm`: $p = 0.97$ and 0.06). Removing a confound turned the count tasks from unmeasurable into measurable, but not into further instances of the late, scale-dependent signal of §5.

Other tasks. Six tasks clear their floors in every model by large margins. `letter_before` clears it in 8 of 16 models at three shots and 3 of 16 at zero shots, and we treat it as marginal throughout. `extraction`'s floor is low (0.171) because position-free letter information says little about which position was asked.

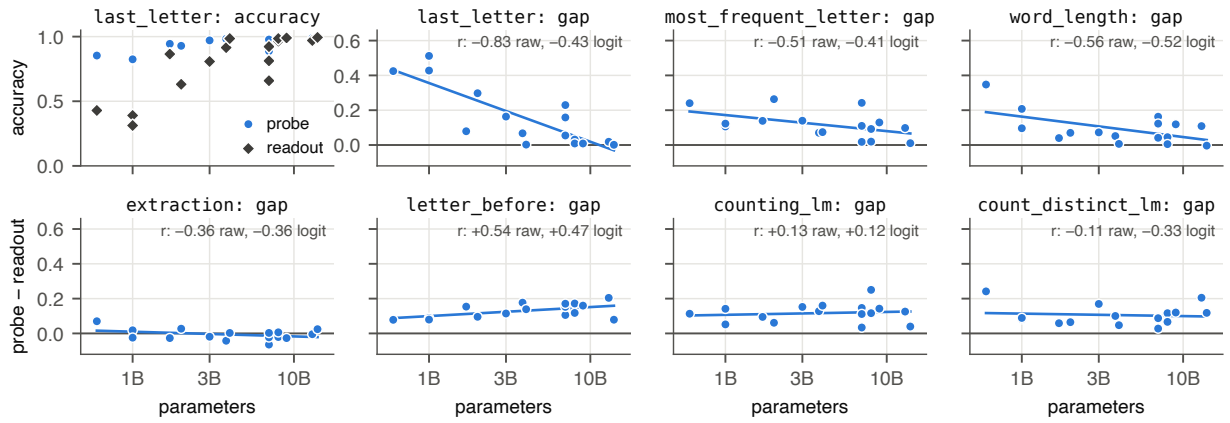


Figure 8: Readout at three shots, one point per model. First panel: `last_letter` probe (held-out-selected) and forced-choice readout accuracy. Other panels: the representation–readout gap (probe − readout) with an OLS line, and its correlation with $\log_{10}$ parameters on the raw and the logit scale. The correlations are uncorrected for the 7×3 tests; permutation p-values, base-only correlations and the mean gap in the three smallest and three largest models are in Table 8.

8 Readout: No Single Scaling Trend

The representation–readout gap is the held-out-selected probe accuracy minus the model's own forced-choice accuracy, at three shots (Figure 8; Table 8).

The `last_letter` narrowing is a ceiling effect. On the raw scale, `last_letter`'s gap falls from +0.455 in the smallest models to +0.009 in the largest ($r = -0.83$). In the largest models the probe is at 0.99–1.00 and readout catches up to it. On the logit scale, which does not compress differences near 1.0, the association is $r = -0.43$ ($p = 0.09$). Our pre-registered criterion required $|r| \geq 0.4$ *and* $p < 0.05$ on the logit scale to keep a narrowing claim, so we make none.

Other tasks disagree with one another. The gap narrows for `word_length` on both scales ($r = -0.52$ on the logit scale, $p = 0.034$), shows no trend for `extraction` (where it is near zero at every size), and grows for `letter_before` ($r = +0.54$, $p = 0.033$; base models only, $r = +0.82$). The two rebuilt count tasks hold a gap of about +0.10 to +0.15 at every scale. With 21 uncorrected tests, none of these individually supports a claim. Together they rule out a single scaling trend, and they do not support the idea that gaps close for retrieval-like tasks and persist for computational ones: `extraction` and `letter_before` are both retrieval-like and behave oppositely. The probe also has hundreds of labelled examples while the model sees three demonstrations, so the gap is not a calibrated measure of unused capacity.

The answer-token subspace. At the final layer the output head separates answers only along the span of the answer tokens' unembedding rows. Projecting the final-layer state onto that span and re-probing beats a random subspace of the same rank by +0.12 (159 of 176 cells, three shots) but beats the top principal components of the same rank by only +0.023. The span is informative, but not much more so than a label-free comparator; it captures a median 6% of the between-class-mean energy. The loss from projection correlates with the readout gap ($r = 0.64$; deficit 0.015, 0.053 and 0.101 across gap tertiles), but because the projected representation is nested in the full one, this association does not show that the span causes the gap. Preliminary runs on a different example sample with a peak-over-layers estimator gave +0.136 and +0.021.

Post-trained models (exploratory). Six of the 16 models are post-trained. At three shots their answer-token span beats the PCA comparator by more than the base models' does (model-level means, difference +0.036, permutation $p = 0.012$), and they lose less accuracy under projection (-0.033, $p = 0.003$); their readout is also higher (0.60 against 0.51). Size does not explain this, since the post-trained models are slightly smaller on average, and at zero shots neither difference is significant ($p = 0.15$). This is consistent with post-training aligning the late representation more closely with the answer-token readout directions. It is also confounded with model family, because the post-trained set is exactly the Qwen3 and Phi-4 models, and the test was not planned in advance. We report it as an observation, not a finding.

9 Discussion

Two sources of character decodability. The data separate two sources of linearly decodable character information. One is stored with the word: present at its tokens in a sentence that asks nothing, largely already in the token embedding, and roughly constant across a more than 20-fold range of model size. The other appears at the answer position only when the property is asked for, mostly in response to that specific property, and in the late half of the network. Only the second grows with scale, so the common observation that larger models "know" more about spelling at the answer position is, in this battery, an observation about the second source.

The late signal is not a relay of the store. If the question merely relayed what a linear probe reads at the word's tokens, the answer position should not systematically exceed them. It falls short of them in small models and exceeds them in large ones, and the difference grows with scale for both tasks where it can be measured. The excess may come from combining information spread over the word's several tokens, from computation the question triggers, or from re-encoding the property into a form a linear probe reads more easily. A linear probe cannot tell these apart, and we avoid the word "routing" because it implies the first reading only.

The early phase. The early bump in the question arm is similar at every scale and is consistent with the word's spelling being carried to the adjacent answer token by early attention. It explains why some question-free signal survives at zero shots and why small models' held-out layer selection can land in the first few layers. It is also a reason to report layer-restricted analyses: a peak over all layers mixes a scale-invariant early signal with a scale-dependent late one.

What the controls changed. Retained question-free signal measured above the majority class was largely nuisance; count tasks that appeared to show a computation gap failed a word-only floor, and length matching alone did not fix them; a strong raw-scale narrowing of the `last_letter` readout gap disappeared on the logit scale. A question placed before the word raised the word tokens' `last_letter` decodability in all 16 models in our main runs, but the pre-registered follow-up shows that this is prefix sensitivity, not anticipation: changing `The word "` to `In the word "` moves the same tokens by a similar amount, and a question adds nothing measurable beyond that. None of these controls is expensive, and each of them changed a conclusion.

10 Limitations

Probes are correlational. All claims concern linear decodability under a held-out-word probe. We do not show that the model uses the store, the late signal or the answer-token span; interventions

that pass modified states through the model would be needed for that. A probe trained on hundreds of labels is not a calibrated upper bound on what the model can do with three demonstrations, and we run no random-label selectivity control.

Linear and positional scope. Absence of linear decodability at a position does not show absence of the information elsewhere or in a nonlinear form. The word-position store is read from the last token or the mean of the word's tokens; other reads could recover more. The store is measured after one prefix (`The word "`), and word-token accuracy shifts by up to about 0.05 with the preceding context. We do not separate single-token from multi-token query words, so we cannot say whether the answer position's excess over the store in larger models requires combining information across a word's tokens. Nor do we fit nonlinear probes at the word position: a small MLP might read a larger store than the linear probe does, in which case the late layers may partly make existing information linearly readable rather than add to it.

Cross-model statistics. The 16 checkpoints come from seven related families with different data, base and post-trained variants, and dense, hybrid and sparse architectures. They are not independent draws, and parameter count is confounded with family and training choices. Within-family slopes and family resampling reduce but do not remove this concern. Each cell uses one sampling seed; a three-seed check on five models in preliminary runs found per-cell standard deviations of at most 0.042. Holm correction covers the four-task claims; the descriptive readout analyses are uncorrected.

Task and format scope. All tasks use English words of length 3–9 from one frequency-ranked vocabulary, single-token direct answers and completion prompts at zero or three demonstrations. Chain-of-thought prompting [Wei et al., 2022, Kojima et al., 2022], character-separated input, other scripts and other tokenizers may behave differently. `letter_before` clears its floor in only half the models, `counting_lm` reuses words (405 unique words for 792 examples) and `count_distinct_lm` tests only above-majority decodability.

Mechanism. We do not identify what produces the late signal or why it exceeds the store in large models.

11 Conclusion

Character properties of a word are linearly decodable from its own tokens in every model we tested, largely from the token embedding and with little change across scale. Asking for a property adds a second, late signal at the answer position, and only this second signal grows with model size; in larger models it carries more linearly decodable information than the word's tokens themselves. Word-only floors, designed-out shortcuts, held-out layer selection and pre-registered tests each removed a claim that a simpler analysis would have made. Character-probing studies should report the word-position baseline, word-only floors and layer-restricted analyses before interpreting a probe at the answer position as knowledge the model has but does not express.

References

Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, Mojan Javaheripi, Piero Kauffmann, James R. Lee, Yin Tat

Lee, Yuanzhi Li, Weishung Liu, Caio C. T. Mendes, Anh Nguyen, Eric Price, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Xin Wang, Rachel Ward, Yue Wu, Dingli Yu, Cyril Zhang, and Yi Zhang. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024. URL https://arxiv.org/abs/2412.08905.

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022. arXiv:2102.12452.

Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the Tuned Lens. *arXiv preprint arXiv:2303.08112*, 2023.

Iaroslav Chelombitko, Ekaterina Chelombitko, and Mika Hämäläinen. Cultural awareness is represented but not decoded: Tracing mythological knowledge across 18 open-source LLMs. *arXiv preprint arXiv:2608.02486*, 2026.

Adrian Cosma, Stefan Ruseti, Emilian Radoi, and Mihai Dascalu. The strawberry problem: Emergence of character-level understanding in tokenized language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 28252–28263, 2025. doi: 10.18653/v1/2025.emnlp-main.1434. URL https://aclanthology.org/2025.emnlp-main.1434/.

Guy Dar, Mor Geva, Ankit Gupta, and Jonathan Berant. Analyzing transformers in embedding space. In *Proceedings of ACL 2023*, 2023. arXiv:2209.02535.

Ayan Datta, Mounika Marreddy, Alexander Mehler, Zhixue Zhao, and Radhika Mamidi. From early encoding to late suppression: Interpreting LLMs on character counting tasks. *arXiv preprint arXiv:2604.00778*, 2026.

Prajit Dhar and Arianna Bisazza. The effect of vocabulary overlap on linguistic probing tasks for neural language models. Poster presented at CLIN 30, 2020.

Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *Transactions of the ACL (TACL)*, 9:160–175, 2021. arXiv:2006.00995.

Gabriel Garcia. The right answer, the wrong direction: Why transformers fail at counting and how to fix it. *arXiv preprint arXiv:2605.03258*, 2026.

Zorik Gekhman, Eyal Ben David, Hadas Orgad, Eran Ofek, Yonatan Belinkov, Idan Szpektor, Jonathan Herzig, and Roi Reichart. Inside-Out: Hidden factual knowledge in LLMs. In *Conference on Language Modeling (COLM)*, 2025.

Gemma Team. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024. URL https://arxiv.org/abs/2408.00118.

Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of EMNLP-IJCNLP 2019*, 2019.

Tatsuya Hiraoka and Kentaro Inui. Spelling-out is not straightforward: LLMs' capability of tokenization from token to characters. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 13340–13353, 2025. doi: 10.18653/v1/2025.findings-emnlp.719. URL https://aclanthology.org/2025.findings-emnlp.719/.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Itay Itzhak and Omer Levy. Models in a spelling bee: Language models implicitly learn the character composition of tokens. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5061–5068, 2022. doi: 10.18653/v1/2022.naacl-main.373.

Ayush Kaushal and Kyle Mahowald. What do tokens know about their characters and how do they know it? In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2487–2507, 2022. doi: 10.18653/v1/2022.naacl-main.179.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.11916.

Henry Kučera and W. Nelson Francis. *Computational Analysis of Present-Day American English*. Brown University Press, Providence, Rhode Island, 1967.

Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

William Merrill, Yanhong Li, Tyler Romero, Anej Svete, Caia Costello, Pradeep Dasigi, Dirk Groeneveld, David Heineman, Bailey Kuehl, Nathan Lambert, Chuan Li, Kyle Lo, Saumya Malik, DJ Matusz, Benjamin Minixhofer, Jacob Morrison, Luca Soldaini, Finbarr Timbers, Pete Walsh, Noah A. Smith, Hannaneh Hajishirzi, and Ashish Sabharwal. Olmo Hybrid: From theory to practice and back. *arXiv preprint arXiv:2604.03444*, 2026. URL https://arxiv.org/abs/2604.03444.

Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of EMNLP 2022*, 2022. arXiv:2202.12837.

Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi. OLMoE: Open mixture-of-experts language models. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2409.02060.

Jane Pan, Tianyu Gao, Howard Chen, and Danqi Chen. What in-context learning "learns" in-context: Disentangling task recognition and task learning. In *Findings of ACL 2023*, 2023. arXiv:2305.09731.

Yoonah Park, Haesung Pyun, and Yohan Jo. Bridging the knowledge–prediction gap in LLMs on multiple-choice questions. In *Proceedings of the International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2509.23782.

Rudolf Rosa, Tomáš Musil, and David Mareček. Measuring memorization effect in word-level neural networks probing. In *Text, Speech, and Dialogue (TSD)*, volume 12284 of *Lecture Notes in Computer Science*, pages 180–188, 2020.

Soma Sato and Ryohei Sasano. How do language models acquire character-level information? In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5987–5997, 2026. doi: 10.18653/v1/2026.eacl-long.282.

Anya Sims, Thom Foster, Klara Kaleb, Tuan-Duy H. Nguyen, Joseph Lee, Jakob N. Foerster, Yee Whye Teh, and Cong Lu. StochasTok: Improving fine-grained subword understanding in LLMs. *arXiv preprint arXiv:2506.01687*, 2025.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Allyson Ettinger, Michal Guerquin, David Heineman, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Jake Poznanski, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 OLMo 2 Furious. In *Proceedings of the Conference on Language Modeling*, 2025. URL https://arxiv.org/abs/2501.00656.

Omri Uzan and Yuval Pinter. CharBench: Evaluating the role of tokenization in character-level tasks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(39):33296–33304, 2026. doi: 10.1609/aaai.v40i39.40615. URL https://arxiv.org/abs/2508.02591.

Sohan Venkatesh. Repeated-token counting reveals a dissociation between representations and outputs. *arXiv preprint arXiv:2605.09239*, 2026.

Elena Voita and Ivan Titov. Information-theoretic probing with minimum description length. In *Proceedings of EMNLP 2020*, 2020. arXiv:2003.12298.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2201.11903.

Jerry Wei, Jason Wei, Yi Tay, Dustin Tran, Albert Webson, Yifeng Lu, Xinyun Chen, Hanxiao Liu, Da Huang, Denny Zhou, and Tengyu Ma. Larger language models do in-context learning differently. *arXiv preprint arXiv:2303.03846*, 2023.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang

Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

Zining Zhu, Jixuan Wang, Bai Li, and Frank Rudzicz. On the data requirements of probing. In *Findings of ACL 2022*, 2022. arXiv:2202.12801.

A Methods

Experiments are run by `scripts/run_v4_suite.sh`, which calls `scripts/run_grid.py` (question arm, readout, accessibility), `scripts/run_wordpos.py` (three prompt arms, word and answer positions, and the stem-order follow-up) and `scripts/run_crosstask.py`. Analyses are `scripts/analyze_v4.py` and `scripts/analyze_v4_followups.py`; every table and figure is generated by `scripts/make_paper_v4.py`.

A.1 Stimuli

Words are lowercase, alphabetic, length 3–9, from a 6000-word frequency-ranked list built from the Brown Corpus [Kučera and Francis, 1967]. Each task draws 800 examples with seed 42; a word unsuitable for a task (a tie for `most_frequent_letter`, no unique preceding letter for `letter_before`) is redrawn. The sampler iterates letters in sorted order, so example sets do not depend on the Python hash seed. Classes with fewer than five examples are dropped before probing.

A.2 Prompts

Question stems, followed by `k` demonstrations of the form `STEM ANSWER.` when shots > 0:

```
The first letter of the word "cat" is
The last letter of the word "cat" is
The 2nd letter of the word "cat" is
Does the word "cat" contain the letter "z"?  Answer yes or no:
In the word "cat", the letter that appears most often is
In the word "cat", the letter immediately before "t" is
The word "cat" has this many letters:
The word "cat" has this many different letters:
In the word "cat", the letter "a" appears this many times:
```

The length-matched tasks use the stems of their base tasks. For digit answers that tokenize as a space followed by the digit, the stem ends with a space so that the digit is the next token. Carriers, with the query term added for term tasks:

neutral `The word "cat" appears in this sentence.`
`The word "cat" and the letter "a" appear in this sentence.`
continuation `Consider the word "cat".  The answer is`

At three shots each carrier is preceded by three neutral sentences about other words, which carry no answers. The answer position is the last non-padding token, located correctly for either padding side. The word position uses the tokenizer's character offsets for the last quoted occurrence of the word: its last overlapping token, or the mean over all overlapping tokens.

A.3 Length-matched count tasks

`counting_lm`. Candidate items are (word, query letter) pairs whose word has exactly two repeated letters (length minus number of distinct letters = 2) and whose count is 1, 2 or 3. At each length, only query letters with at least two items at every count are kept. The design rectangle is the widest contiguous range of lengths in which every (length, count) cell has at least 20 items: lengths 6–9, counts 1–3. Each of the 12 cells receives $\lfloor 800/12 \rfloor = 66$ examples (792 in total); within a cell the query letter is drawn uniformly from the kept letters, then an item with that letter. This yields 405 unique words.

`count_distinct_lm`. Words are binned by (length, number of distinct letters); the rectangle with the most answer classes (then the most lengths) in which every cell has at least 20 words is lengths 8–9 and answers 5–8, with 100 examples per cell (674 unique words).

Word-only predictors at three shots for `counting_lm`, against a 0.271 majority: length 0.251, repetition (length minus distinct letters) 0.271, query letter 0.222, letter set 0.404. For the intermediate, length-matched-only design they were 0.273, 0.594 and 0.46 for the first three, with a floor of 0.611.

A.4 Probe and estimator

The probe is multinomial logistic regression on standardized features, fit with L-BFGS ($\ell_2 = 10^{-3}$, at most 200 iterations) on the GPU. Folds are three groups of unique query words assigned by a fixed permutation, and each fold's fit starts from zero (cold start). Twelve layers are probed per model, evenly spaced from the embedding layer to the last layer. Held-out layer selection chooses, for each test fold, the layer with the best mean accuracy on the other two folds, and averages the resulting test accuracies.

Estimator check. Preliminary runs used the maximum over layers at an absolute stride of four (so deeper models had more layers to choose from) and warm-started each fold from the previous fold's solution. Under the present estimator, the per-task mean selection optimism (peak minus held-out-selected accuracy) ranges from 0.001 to 0.019, and its correlation with model depth ranges from -0.57 to $+0.48$ across tasks, with most values negative. Representation accuracy at three shots differs from the preliminary estimator's by -0.021 to $+0.004$ per task. Scaling results are therefore not an artefact of layer selection or warm starts.

A.5 Nuisance floors

For each task and shot count we rebuild the exact example set and fit, on the same folds, predictors that see only the word: length (per-length modal answer), the 26-dimensional binary letter set, the letter set plus a one-hot query letter (term tasks), and letter set plus length (with the query letter for term tasks), each a logistic regression on standardized features. The floor is the best non-oracle predictor, or the majority baseline if none beats it. Oracles are length for `word_length` and the letter set (with or without length) for the two `count_distinct` tasks.

A.6 Accessibility

At the final layer, after the model's final normalization, we take the mean-centred unembedding rows of the kept answer tokens, form an orthonormal basis Q of their span (rank $k-1$ for k classes), and re-probe $\tilde{h}QQ^\top$. Comparators are three random orthonormal subspaces of the same rank and the top principal components of the same rank, computed without labels.

B Pre-Registered Criteria and Outcomes

Decision criteria were written into the project's analysis plan before the corresponding data existed. All use held-out layer selection.

B.1 Word-position experiment

Criterion. For `last_letter`, `most_frequent_letter` and `first_letter` at zero shots: (i) the word-position store in the neutral sentence is above the nuisance floor in at least 12 of 16 models; (ii) the slope of (answer position under the question − store) against $\log_{10}$ parameters is positive with permutation $p < 0.05$. **Outcome.** (i) holds in 16 of 16 models for all three tasks under both word reads. (ii) holds for `last_letter` ($p = 0.010$ span mean, 0.017 last token) and `most_frequent_letter` ($p = 0.012$ and 0.011). It fails for `first_letter` ($p = 0.40$ and 0.49), whose answer position is at ceiling in every model (above-floor accuracy +0.537 against a maximum of +0.543).

B.2 Secondary criteria

- **Readout gap.** Keep a narrowing claim for `last_letter` only if the logit-scale correlation has $|r| \geq 0.4$ and $p < 0.05$. Outcome: $r = -0.43$, $p = 0.09$; claim dropped (§8).
- **Estimator.** Restate preliminary results if mean selection optimism exceeds 0.03 and correlates with depth at $r > 0.5$. Outcome: 0.001–0.019, correlations mostly negative; no restatement needed (Appendix A.4).
- **Count tasks.** Reinstate a count-task comparison if `counting_lm` clears its floor in at least 8 of 16 models. Outcome: 14 of 16 at three shots, 16 of 16 at zero shots (§7).
- **Interaction.** Report the question×scale slope for all four prompt-sensitive tasks whatever the result, and claim only those with $p < 0.05$. Outcome: all four, at both shot counts (Table 4).

B.3 Stem-order follow-up (null result)

In the main runs, a question placed before the word (`The last letter of the word "cat" is`) raised the word tokens' `last_letter` decodability relative to the neutral sentence in 16 of 16 models (+0.062). Prefixes differed between the two prompts, so we tested word order directly at zero shots with three prompts per example: `qfirst` (`The last letter of the word "cat" is`), `wfirst` (`In the word "cat", the last letter is`) and the neutral sentence, with matching templates for `first_letter`, `most_frequent_letter` ("most frequent letter") and `word_length` ("number of letters").

Criteria. Primary: for `last_letter` and `most_frequent_letter`, word@`qfirst` − word@`wfirst` > 0 in at least 12 of 16 models with sign-test $p < 0.05$ (last-token read). Prefix control: word@`wfirst` − word@neutral has mean $|\Delta| < 0.02$, since in `wfirst` the word precedes any question.

Outcome: fail. The primary prediction fails for `last_letter` (11 of 16, $p = 0.21$) and passes for `most_frequent_letter` (15 of 16, $p = 0.0005$), but the prefix control fails for both (+0.041 and +0.020). The word is tokenized identically in all three prompts (checked with the Qwen3 tokenizer), so under causal attention the word's tokens differ only in the preceding text: `The last letter of the word "`, `In the word "`, or `The word "`. A change from `The` to `In the` moves the word's encoding as much as the question does. No `qfirst` advantage grows with scale.

task	read	qfirst−wfirst	positive	sign p	wfirst−neutral	r vs scale (p)
last_letter	last token	+0.021	11/16	0.210	+0.041	−0.12 (0.660)
last_letter	span mean	+0.039	15/16	0.0005	+0.051	+0.07 (0.803)
most_frequent_letter	last token	+0.019	15/16	0.0005	+0.020	+0.20 (0.458)
most_frequent_letter	span mean	+0.013	11/16	0.118	+0.015	+0.07 (0.797)
first_letter	last token	+0.007	11/16	0.057	−0.014	+0.32 (0.240)
first_letter	span mean	+0.002	10/16	0.302	+0.004	+0.13 (0.622)
word_length	last token	+0.089	16/16	<0.0001	+0.063	+0.38 (0.148)
word_length	span mean	+0.077	16/16	<0.0001	+0.078	+0.40 (0.123)

Table 1: Stem-order follow-up, zero shots, 16 models, held-out layer selection. Columns give the mean difference, models with a positive difference, a two-sided exact sign test, the prefix control (wfirst − neutral), and the correlation of the difference with scale.

C Measurement Hygiene

Four measurement problems found during development shaped the present protocol.

Stimulus-pool size. On a pool of about 100 words, the same model and probe gave a memorization false positive under random splits (most_frequent_letter at 1.000 for Qwen3-8B, peaking at layer 2) and an underpowered false negative under held-out-word splits (0.358 against a 0.309 majority, with about 26 training words per fold). The 6000-word pool with held-out-word folds gives 0.845 against 0.256. The regimes also differed in example count and layer stride, so this is a sensitivity demonstration rather than a calibrated pool-size effect; it is why every result here uses the large pool.

Padding side. Answer positions were originally located as the attention-mask sum minus one, which is correct only for right-padded batches. Gemma-2's tokenizer pads on the left, so its answer-position states were read from the wrong token. An unbatched free-generation check exposed this: Gemma-2-9B scored 0.098 on first_letter under batched scoring and 1.000 under free generation. The error grew monotonically with shot count, as longer prompts mean more padding. The current code locates the last real token for either padding side, and all Gemma-2 numbers here come from corrected runs.

Free-generation scoring. The free-generation check counts a hit if the target starts or appears early in a short generation. That is informative for 22–26-class letter answers but not for two- or three-class answers, where almost any generation containing a digit scores. We therefore use free generation only as a diagnostic for letter tasks and report forced-choice readout everywhere.

Sampler determinism. An initial version of the sampler iterated Python sets when choosing query letters, so four tasks depended on the hash seed. The sampler now iterates sorted sets, and analysis code orders models by size and then name, so all reported numbers are identical across hash seeds.

D Result Tables

Exact values behind the figures in the main text, generated from the analysis outputs by scripts/make_paper_v4.py.

family	sizes	notes
Qwen3	0.6B, 1.7B, 4B, 8B	post-trained releases
Llama-3.x	3.2-1B, 3.2-3B, 3.1-8B	base
Gemma-2	2B, 9B	base; right padding enforced (Appendix C)
OLMo-2	1B, 7B, 13B	base
Phi-4	3.8B-mini-instruct, 14B	post-trained
OLMo-Hybrid	7B	DeltaNet/attention hybrid
OLMoE	1B-7B	sparse MoE, plotted at 7B total parameters

Table 2: The 16 models. Every experiment covers all 16.

task	shots	floor	word, last token	word, span mean	answer, no question	answer, question
`first_letter`	0	0.457	+0.434 (16)	+0.508 (16)	+0.436 (16)	+0.537 (16)
`first_letter`	3	0.475	+0.415 (16)	+0.492 (16)	+0.396 (16)	+0.522 (16)
`last_letter`	0	0.547	+0.314 (16)	+0.177 (16)	+0.115 (16)	+0.356 (16)
`last_letter`	3	0.552	+0.296 (16)	+0.157 (16)	−0.004 (6)	+0.393 (16)
`most_frequent_letter`	0	0.442	+0.135 (16)	+0.165 (16)	+0.084 (14)	+0.283 (16)
`most_frequent_letter`	3	0.400	+0.134 (16)	+0.162 (16)	+0.018 (10)	+0.357 (16)
`word_length`	0	0.363	+0.003 (7)	−0.048 (2)	+0.029 (14)	+0.193 (12)
`word_length`	3	0.351	+0.019 (10)	−0.018 (4)	+0.062 (15)	+0.383 (16)
`count_distinct_lm`	0	0.216	+0.077 (16)	+0.088 (16)	+0.072 (16)	+0.079 (16)
`count_distinct_lm`	3	0.222	+0.100 (16)	+0.090 (16)	+0.066 (16)	+0.098 (16)

Table 3: Accuracy above the word-only nuisance floor (held-out layer selection), mean over the 16 models, with the number of models above the floor in parentheses (Figure 2). "Word" columns probe the query word's tokens in the neutral sentence (last token, or mean over the word's tokens); "answer" columns probe the final prompt token. For `count_distinct_lm` the floor equals the majority baseline because the letter set, which determines the answer, is excluded as an oracle.

		all layers, held-out-selected				late ($\geq$0.5)		early ($\leq$0.25)	
task	shots	slope	p (Holm)	within-fam.	family CI	slope	p_{Holm}	slope	p
`last_letter`	0	+0.126	0.0005 (0.002)	+0.125	[+0.053, +0.159]	+0.162	0.0006	−0.014	0.691
`most_frequent_letter`	0	+0.192	0.0006 (0.002)	+0.173	[+0.084, +0.241]	+0.202	0.0006	−0.003	0.905
`extraction`	0	+0.140	0.0001 (0.0004)	+0.160	[+0.089, +0.183]	+0.138	0.0004	−0.011	0.435
`letter_before`	0	+0.094	0.015 (0.015)	+0.104	[+0.021, +0.170]	+0.091	0.015	+0.004	0.710
`last_letter`	3	+0.132	0.0008 (0.003)	+0.129	[+0.056, +0.186]	+0.151	0.004	−0.005	0.924
`most_frequent_letter`	3	+0.171	0.001 (0.003)	+0.157	[+0.091, +0.266]	+0.205	0.0008	−0.002	0.948
`extraction`	3	+0.156	0.009 (0.009)	+0.192	[+0.093, +0.271]	+0.151	0.014	−0.007	0.672
`letter_before`	3	+0.135	0.002 (0.004)	+0.145	[+0.079, +0.197]	+0.131	0.003	−0.003	0.837

Table 4: Question×scale interaction (Figure 1c): slope per decade of parameters of (question arm − neutral arm) at the answer position. "All layers" uses held-out layer selection; the permutation p is followed by its Holm-adjusted value across the four tasks. "Late" and "early" restrict each arm to layers at relative depth ≥ 0.5 or ≤ 0.25 (maximum over the restricted layers); the late p is Holm-adjusted across the four tasks. Family CI: 95% interval from resampling model families.

	shots = 0			shots = 3		
task	majority	neutral	continuation	majority	neutral	continuation
`extraction`	0.228	−0.036	0.240	0.151	−0.000	0.017
`last_letter`	0.671	0.324	0.473	0.503	−0.009	0.034
`most_frequent_letter`	0.542	0.295	0.467	0.324	0.051	0.104
`letter_before`	0.485	–	–	0.283	–	–
`word_length`	0.546	0.149	0.439	0.416	0.162	0.175
`counting_lm`	0.698	0.520	0.859	0.768	0.479	0.783
`count_distinct_lm`	0.911	0.911	1.082	0.673	0.673	0.610
`first_letter`	0.885	0.811	0.855	0.856	0.758	0.696
`contains_letter`	0.966	0.919	1.233	0.837	0.635	0.866

Table 5: Retained fraction of the question arm's signal when the question is deleted (Figure 5): mean(no-question − base) / mean(question − base). "Majority" uses the majority baseline and the neutral carrier; the other columns use the word-only floor. Missing values: the question arm's mean is not above the floor (`letter_before`).

probed label	total Δ	generic frame	wrong-property Q	specific Q	cells	models
`last_letter`	0.241	22%	8%	**70%**	32	16
`most_frequent_letter`	0.228	22%	38%	**40%**	28	14
`word_length`	0.296	28%	25%	**47%**	30	10
`first_letter`	0.137	47%	32%	**21%**	20	10
`extraction`	0.221	34%	−5%	**71%**	32	16
`letter_before`	0.142	29%	9%	**62%**	28	14

Table 6: Decomposition of the question effect at zero shots (Figure 6), zero-leak cells, all 16 models, held-out layer selection. Shares are of the total Δ (self − neutral, above-majority scale). Below the rule: the secondary term-label arm. The continuation floor is joined per (model, set, task) from the word-position runs, whose example draws differ from the cross-task runs'.

task	majority	floor	floor predictor	models above floor	best above floor
`first_letter`	0.123	0.475	length + letter set	16/16	+0.525
`last_letter`	0.145	0.552	length + letter set	16/16	+0.447
`extraction`	0.122	0.171	length + letter set	16/16	+0.503
`most_frequent_letter`	0.256	0.400	length + letter set	16/16	+0.483
`contains_letter`	0.476	0.678	length + letter set	16/16	+0.272
`word_length`	0.185	0.351	letter set	16/16	+0.635
`letter_before`	0.128	0.416	length + letter set	8/16	+0.187
`counting`	0.311	0.676	length + letter set	2/16	+0.094
`count_distinct`	0.289	0.532	length	3/16	+0.033
`counting_lm`	0.271	0.404	letter set	14/16	+0.259
`count_distinct_lm`	0.222	0.222	majority	16/16	+0.192

Table 7: Word-only nuisance floors at three shots and the number of models whose question-arm probe (answer position, held-out layer selection) exceeds the floor (Figure 7). "Best above floor" is the maximum over the 16 models. Below the rule: the original count tasks and their designed replacements.

task	gap, 3 smallest	gap, 3 largest	r (raw)	r (logit)	r (base only, raw)
`last_letter`	+0.455	+0.009	−0.83 (0.0002)	−0.43 (0.091)	−0.93 (0.0003)
`most_frequent_letter`	+0.157	+0.079	−0.51 (0.047)	−0.41 (0.116)	−0.26 (0.454)
`word_length`	+0.217	+0.075	−0.56 (0.019)	−0.52 (0.034)	−0.26 (0.468)
`extraction`	+0.021	−0.002	−0.36 (0.174)	−0.36 (0.174)	−0.39 (0.269)
`letter_before`	+0.082	+0.148	+0.54 (0.033)	+0.47 (0.064)	+0.82 (0.003)
`counting_lm`	+0.102	+0.102	+0.13 (0.624)	+0.12 (0.659)	+0.24 (0.497)
`count_distinct_lm`	+0.137	+0.148	−0.11 (0.699)	−0.33 (0.227)	+0.33 (0.353)

Table 8: Representation–readout gap at three shots (Figure 8): mean over the three smallest (Qwen3-0.6B, Llama-3.2-1B, OLMo-2-1B) and the three largest (phi-4, OLMo-2-13B, Gemma-2-9B) models, and its correlation with $\log_{10}$ parameters (permutation p). The logit column uses logit(probe) − logit(readout), which removes the compression near 1.0. No correction for the 7×3 tests is applied.